Project 10 — Autoencoder Anomaly Detection

Level: Advanced | **Dataset:** Credit Card Fraud (Kaggle) | **Framework:** PyTorch

---

Objective

Build an Autoencoder that learns the distribution of normal transactions and flags anomalies via high reconstruction error.
Cover: encoder-decoder architecture, bottleneck, reconstruction loss, threshold tuning, unsupervised anomaly detection.

Project Structure

```
10_autoencoder_anomaly/
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_preprocessing.ipynb
│   └── 03_train_evaluate.ipynb
├── data/raw/creditcard.csv
├── data/processed/
├── models/model.pkl
├── charts/
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Dataset: `mlg-ulb/credit-card-fraud-detection` — Kaggle. 284,807 transactions, 492 fraud (0.17% fraud rate). Features: V1-V28 (PCA-transformed), Amount, Time. Target: Class (0=normal, 1=fraud).

Notebook 01 — EDA (`01_eda.ipynb`)

STOP 1 — Load & Class Distribution

- Load creditcard.csv
- Print class distribution: 284,315 normal, 492 fraud
- This is extreme imbalance: 99.83% vs 0.17%
- **Agent stops here. Explain:**
  - Why this is a perfect Autoencoder use case: we have very few fraud examples
  - The unsupervised insight: train ONLY on normal → AE learns to reconstruct normal well → fraud reconstructed poorly
  - Why supervised models struggle here: too few fraud examples even with class weights
  - Real-world context: in production, fraud patterns change constantly — unsupervised is more robust
- Wait for user confirmation before continuing

STOP 2 — Feature Analysis

- Plot distribution of `Amount` — heavily skewed, log transform
- Plot distribution of V1, V5, V14 (most fraud-discriminative PCA components)
- Overlay normal vs fraud distributions for V14
- **Agent stops here. Explain:**
  - What V1-V28 are: principal components of original transaction features (anonymized by Kaggle)
  - Why Amount needs special treatment (raw dollar amount vs scaled PCA features)
  - What overlapping distributions mean: fraud and normal transactions look similar to linear models
  - How the AE exploits the subtle difference: it learns the joint distribution of all features
- Wait for confirmation

STOP 3 — Reconstruction Concept Walkthrough

- Explain (with markdown cells) what reconstruction means:
  - AE takes input x → compresses to z (bottleneck) → reconstructs x̂
  - Loss: MSE(x, x̂) averaged over all features
  - At inference: high MSE = anomalous
- **Agent stops here. Explain:**
  - The information bottleneck principle: compression forces the model to learn the "essence"
  - Why bottleneck width matters: too wide = AE memorizes everything (no anomaly detection), too narrow = loses normal patterns too
  - What reconstruction error distribution looks like for normal vs fraud
- Wait for confirmation

Notebook 02 — Preprocessing (`02_preprocessing.ipynb`)

STOP 4 — Isolation of Normal Class

- Separate: `normal_df = df[df['Class'] == 0]`
- Training set: 80% of normal only (no fraud in training)
- Validation set: 10% normal + ALL 492 fraud (to tune threshold)
- Test set: remaining 10% normal + reserved 100 fraud
- **Agent stops here. Explain:**
  - Why we train ONLY on normal data — this is the core principle of AE-based anomaly detection
  - Why we include fraud in validation: to find the optimal reconstruction error threshold
  - The correct split strategy for unsupervised anomaly detection
- Wait for confirmation

STOP 5 — Feature Scaling

- Log transform `Amount`: `np.log1p(df['Amount'])`
- Drop `Time` column (not informative after PCA)
- `StandardScaler` fit on normal train features only
- Apply to normal train, val (normal+fraud), test (normal+fraud)
- **Agent stops here. Explain:**
  - Why log1p for Amount: log(1+x) handles zero correctly, compresses large values
  - Why StandardScaler fit only on normal train: we're assuming normal distribution of normal transactions
  - What happens if we scale fraud using fraud statistics (leakage, defeats the purpose)
- Wait for confirmation

STOP 6 — Tensor Dataset

- Normal train: X only (no labels needed for training — unsupervised)
- Val/Test: (X, y) pairs where y is the fraud label for evaluation
- DataLoader for train: batch_size=256, shuffle=True
- **Agent stops here. Explain:**
  - Why training DataLoader has no labels: AE is trained to minimize reconstruction error, not classify
  - How this is fundamentally different from all previous supervised projects
  - What "unsupervised learning" means in practice
- Wait for confirmation

Notebook 03 — Train & Evaluate (`03_train_evaluate.ipynb`)

STOP 7 — Autoencoder Architecture

```
Encoder:
  Linear(29, 64) → ReLU → Dropout(0.1)
  Linear(64, 32) → ReLU
  Linear(32, 16) → ReLU  [bottleneck = 16]

Decoder:
  Linear(16, 32) → ReLU
  Linear(32, 64) → ReLU
  Linear(64, 29)          [no activation — reconstruct any value]
```

Forward: `x → z = encode(x) → x_hat = decode(z) → return x_hat`

- **Agent stops here. Explain:**
  - Symmetric encoder-decoder: decoder mirrors encoder structure
  - Bottleneck dimension=16: compresses 29 features to 16 (forced information bottleneck)
  - Why no activation at decoder output: output must match input range (any real value after scaling)
  - What the latent space z represents: compressed representation of the transaction
  - How to choose bottleneck size: experiment — too small loses normal patterns, too large = no compression
- Wait for confirmation

STOP 8 — Reconstruction Loss

- Use `nn.MSELoss(reduction='none')` — keep per-sample, per-feature losses
- Average over features for per-sample reconstruction error
- Training loss: mean of per-sample errors
- **Agent stops here. Explain:**
  - Why `reduction='none'`: we need per-sample error at inference time
  - What reconstruction error for ONE sample looks like: scalar value (mean over 29 features)
  - Why MSE penalizes large reconstruction errors quadratically — good for detecting anomalies
  - Alternative: MAE loss — less sensitive to outliers (sometimes better for AE)
- Wait for confirmation

STOP 9 — Training Loop

- Train on NORMAL ONLY for 50 epochs
- Track train reconstruction error per epoch
- Also compute val reconstruction error for normal vs fraud separately
- Plot: normal reconstruction error distribution vs fraud reconstruction error distribution
- **Agent stops here. Explain:**
    - What we expect to see: two distributions, fraud shifted right (higher error)
    - Why the distributions might overlap: some fraud looks like normal, some normal looks weird
    - The separation quality directly predicts AUC
    - What "collapse" looks like if bottleneck is too wide: both distributions identical
- Wait for confirmation

STOP 10 — Threshold Tuning

- Compute reconstruction error for ALL validation samples (normal + fraud)
- Try thresholds from min to max error at 100 steps
- For each threshold: compute Precision, Recall, F1
- Plot F1 vs threshold curve
- Select threshold that maximizes F1 (or recall, depending on business requirement)
- **Agent stops here. Explain:**
    - What threshold selection is: converting a continuous score to binary prediction
    - The precision-recall tradeoff at different thresholds
    - In fraud detection, what is worse: false positive (block good transaction) vs false negative (miss fraud)?
    - Why we tune on val, evaluate on test (never touch test during tuning)
- Wait for confirmation

STOP 11 — Evaluation on Test Set

- Apply tuned threshold to test set
- Compute: Precision, Recall, F1, AUC-ROC, AUC-PR
- Plot ROC curve and Precision-Recall curve
- **Agent stops here. Explain:**
  - Why AUC-PR is more informative than AUC-ROC for extreme imbalance
  - What AUC-PR = 0.5 means on a 0.17% fraud rate (baseline = 0.0017!)
  - Why ROC can be misleadingly optimistic with extreme imbalance
  - The business metric: catch rate (recall on fraud) at a given false positive rate
- Wait for confirmation

STOP 12 — Latent Space Visualization

- Encode all test samples (normal + fraud) to get z vectors [N, 16]
- Apply t-SNE or PCA to reduce to 2D
- Plot with color: blue=normal, red=fraud
- **Agent stops here. Explain:**
  - What we hope to see: fraud forming clusters away from normal
  - What t-SNE shows that PCA doesn't: non-linear clustering structure
  - Why fraud might not perfectly separate in latent space (some fraud IS similar to normal transactions)
  - How this visualization helps in understanding model failure modes
- Wait for confirmation

STOP 13 — Save & Inference

- Save model.state_dict(), scaler, threshold
- Write `predict_fraud(transaction_dict)` → label, reconstruction_error, is_fraud
- **Agent stops here. Explain:**
  - Complete inference pipeline: dict → preprocess (log Amount, scale) → tensor → model.eval() → reconstruct → MSE → compare to threshold → return
  - Why we save the threshold with the model (it's part of the "model")
  - How to update threshold in production as fraud patterns evolve
- Wait for confirmation

`dashboard_core.py`

Functions:

- `load_model_scaler_threshold()` → model, scaler, threshold
- `predict_fraud(transaction_dict)` → reconstruction_error, is_fraud, bool
- `get_error_distributions()` → (normal_errors, fraud_errors) arrays
- `get_roc_pr_curves()` → dict of curve data
- `get_latent_viz()` → 2D coords + labels

`app.py` — Streamlit (~80 lines)

Sections:

1. Sidebar: sliders for V1, V14, V17, Amount (most discriminative features)
2. Main: "Analyze Transaction" → show reconstruction error + fraud/normal verdict
3. Tab 1: Training reconstruction error curve
4. Tab 2: Error distribution histogram (normal vs fraud overlap)
5. Tab 3: ROC + PR curves

Key Concepts Covered

- Autoencoder architecture (encoder, bottleneck, decoder)
- Information bottleneck principle
- Training on normal only (unsupervised anomaly detection)
- Reconstruction loss (MSE reduction='none' for per-sample)
- Threshold tuning on validation set
- AUC-PR vs AUC-ROC for imbalanced data
- Latent space visualization with t-SNE
- Full unsupervised learning pipeline